

Módulo "Quiebre Dinámico"

Alertar antes de que un producto llegue a cero en el punto de venta, no cuando ya llegó — línea base estadística por percentiles, detección de tendencia y ajuste por urgencia temporal según la próxima visita programada.

En una frase: compara el inventario actual de cada producto contra su propio comportamiento histórico "normal" en ese tipo de PDV, y avisa cuando la tendencia dice que va a llegar a 0 **antes de que alguien vuelva a visitarlo** — no simplemente "está bajo".

Documento generado: 15 de agosto de 2026

Estado del módulo: Capa 1 y Capa 2 desplegadas en producción, recálculo automático enganchado al scheduler (línea base 1x/día, alertas cada 4h), pantalla propia en el portal (Quiebre Dinámico, roles admin/analyst)

Contiene una dependencia de datos crítica sin resolver: ver sección 8

1. Resumen ejecutivo

Hoy, cuando un producto se agota en un punto de venta, el sistema lo registra recién cuando ya pasó: `BALANCES_TOTALES.estado_producto = 'quiebre'` es un dato del pasado, no una alerta. Para cuando alguien lo ve, el producto ya estuvo fuera de anaquel un tiempo indeterminado, y la próxima oportunidad de reponerlo es la siguiente visita programada — que puede ser en días.

"Quiebre Dinámico" resuelve esto con dos capas:

CAPA 1 · NOCTURNA Línea base por percentiles Por cada grupo (categoría × tipo de PDV × frecuencia × cliente), calcula qué nivel de inventario es "normal" a partir de los últimos 90 días.	CAPA 2 · CADA RECÁLCULO Señal de riesgo + urgencia Compara el inventario actual contra esa línea base y la tendencia reciente, y decide si va a llegar a 0 antes de la próxima visita.	RESULTADO Alerta accionable Solo se guardan los casos con riesgo real (Alto/Medio) — no duplica lo que <code>estado_producto</code> ya muestra.
--	--	---

El módulo está desplegado en producción con datos reales: **146 grupos de línea base calculados, 773 alertas activas, 78 marcadas urgentes** (corte del 15 de agosto de 2026). Sin embargo, depende de que **FRECUENCIAS_PDVS_CLIENTE** esté cargado para agrupar y priorizar bien, y hoy solo cubre el 4,8% de las combinaciones activas — detallado en la sección 8.

2. Objetivo de negocio

El objetivo es anticipar, no solo registrar:

1. **Detectar la tendencia hacia el quiebre** antes de que el producto llegue a 0, comparando contra el comportamiento normal de ese tipo de PDV — no un umbral fijo igual para todos.
2. **Priorizar por urgencia temporal real** — un producto bajo en un PDV que se visita mañana no es lo mismo que uno bajo en un PDV que se visita en 5 días. Solo el segundo caso se marca **urgente** .

El módulo no decide reposición ni genera pedidos automáticamente. Es una señal para que el analista de cuenta priorice — hoy ese trabajo se hace en Excel, de forma manual y sin visibilidad de tendencia.

3. Capa 1 — Línea base por percentiles

3.1 Universo de datos

Se usan las filas de `BALANCES_TOTALES` con `estado_producto = 'normal'` y `caras` no nulo, de los últimos **90 días**. Se excluyen a propósito las filas ya marcadas `quiebre`: si se incluyeran, contaminarían el cálculo de "qué es normal" con los propios casos que se quieren detectar.

3.2 Por qué percentiles, no promedio $\pm$ desviación estándar

Decisión de diseño: el inventario en un punto de venta es un diente de sierra — sube de golpe en cada reposición y baja hasta la próxima visita — no una distribución estable alrededor de un promedio. Con datos de conteo (caras van de 0 a un puñado de unidades), `promedio - desviación` puede dar umbrales negativos sin sentido. Los percentiles (P10, P25) no tienen ese problema y describen mejor "qué tan bajo es bajo de verdad" para ese grupo específico.

3.3 Agrupación

Los percentiles se calculan por combinación de:

- `id_categoria` — categoría del producto
- `PUNTOS_INTERES1.jerarquia_nivel_2` — tipo de PDV (ej. "Farmacia", "Automercado")
- `bucket_frecuencia` — ver 3.4
- `id_cliente`

3.4 `bucket_frecuencia`: por qué agrupar en 4 rangos y no usar el valor exacto

La frecuencia semanal se agrupa en 4 buckets (`$\geq 3$ /semana` , `1-2/semana` , `quincenal` , `mensual`) en vez de usar `frecuencia_semanal` tal cual. Con el valor exacto, los grupos de baja frecuencia (mensuales, por ejemplo) quedan con muy pocas muestras para calcular un percentil confiable — agrupar en rangos más anchos junta suficiente historial sin perder la distinción que de verdad importa (un PDV que se visita 1 vez al mes tiene un patrón de inventario muy distinto a uno que se visita 3 veces por semana).

3.5 Piso mínimo de muestras

MIN_MUESTRAS_GRUPO = 5. Un P10/P25 calculado sobre 1 o 2 muestras no es un percentil confiable, es ruido. Los grupos con menos de 5 balances en la ventana de 90 días se descartan directamente de la línea base — esas combinaciones quedan sin línea base y, por diseño, sus productos nunca disparan riesgo (solo "Normal") hasta que acumulen suficiente historial.

3.6 Resultado real en producción

MÉTRICA	VALOR OBSERVADO (15 AGO 2026)
Balances "normal" con caras, últimos 90 días	17.582
Grupos calculados (≥5 muestras)	146
Grupos descartados por poca data	74

Casi el 34% de los grupos candidatos no llegó al piso mínimo de muestras — esperable con solo 90 días de ventana en un sistema todavía en fase de adopción, y es exactamente el comportamiento de "no alcanza el dato todavía" que se prefiere sobre inventar un umbral poco confiable.

4. Capa 2 — Señal de riesgo

4.1 Tendencia

Para cada combinación (PDV, producto, cliente), se toman las **2 lecturas más recientes** de `BALANCES_TOTALES` (vía `ROW_NUMBER()` particionado por esa combinación, ordenado por fecha descendente):

```
tendencia = caras_actual - caras_anterior
```

4.2 Clasificación de riesgo

CONDICIÓN	RIESGO
<code>caras_actual ≤ 0</code>	Quiebre (ya existe como <code>estado_producto</code>)
<code>caras_actual ≤ P10</code> y <code>tendencia < 0</code>	Riesgo ALTO
<code>caras_actual ≤ P25</code> y <code>tendencia ≤ 0</code>	Riesgo MEDIO
Ninguna de las anteriores	Normal

4.3 Qué se persiste, y por qué "Quiebre" no

Decisión de diseño confirmada contra datos reales: solo se guardan en `ALERTAS_QUIEBRE` las filas con riesgo ALTO o MEDIO. Las de riesgo "Quiebre" (`caras_actual ≤ 0`) no se persisten — ya son visibles tal cual en `BALANCES_TOTALES.estado_producto` , y guardarlas también acá solo infla la tabla sin sumar información nueva: se confirmó contra datos reales que ~80% de las combinaciones activas ya están en 0 en un momento dado, así que "Quiebre" no es una señal escasa que valga la pena duplicar.

5. Ajuste por urgencia temporal

Esta es la parte que responde de verdad a "alertar antes de que pase":

```
días_hasta_próxima_visita = próxima fecha en RUTA_PROGRAMACION - hoy
días_para_llegar_a_cero    = caras_actual / |tendencia|    (solo si tendencia < 0)

SI riesgo ≠ Normal Y ambos valores existen Y días_para_llegar_a_cero ≤
días_hasta_próxima_visita
  → URGENTE: va a quebrar antes de la próxima visita programada
```

La prioridad del PDV no toca esta matemática. Un PDV de baja prioridad no "necesita caer más" para disparar la alerta — eso escondería quiebres reales de clientes o PDVs que hoy tienen prioridad baja pero igual dependen de esta reposición. La prioridad de `RUTA_PROGRAMACION` se reserva para **ordenar** qué alerta se le muestra primero al analista, no para calcular si hay riesgo.

Estado real hoy: ese ordenamiento por prioridad todavía no está implementado en el listado — el orden actual es `urgente DESC, riesgo ASC, días_para_llegar_a_cero ASC`. Ver sección 9.

6. Por qué NO Machine Learning acá

El roadmap de este módulo evaluó explícitamente empezar con un modelo entrenado y se decidió que no, por ahora:

- **Explicabilidad:** la fórmula estadística es completamente auditable — cualquiera puede recalcular a mano por qué una alerta disparó. Un modelo de caja negra no da esa garantía sin trabajo adicional (SHAP, etc.).
- **Validación directa:** ya existe un ground truth real (`estado_producto = 'quiebre'`) contra el cual comparar si el sistema hubiera alertado a tiempo, sin necesidad de esperar a etiquetar nada.
- **Velocidad de implementación:** sin entrenamiento, sin gestión de modelo, sin reentrenar — la fórmula corre igual desde el primer día que hay suficiente historial (sección 3.5).

Cuándo sí tendría sentido un modelo: un modelo de series de tiempo por SKU-PDV (roadmap predictivo, ítem S1) es la evolución natural de este módulo — pero recién *después* de que esta fórmula acumule histórico suficiente, porque ese histórico es exactamente el que serviría como features y labels para entrenarlo. Ver también sección 14 de la ficha de Plan de Acción, donde el mismo patrón (empezar simple, entrenar después con datos propios) ya se aplicó con éxito.

7. Incidentes y hallazgos técnicos del desarrollo

Incidente real en producción — NaN al calcular alertas. Al correr por primera vez `POST /alertas/recalcular` contra producción, falló con `"cannot convert float NaN to integer"`. Causa: algunas filas de `BALANCES_TOTALES` tienen `id_product` nulo (producto no identificado bien al escanear la etiqueta con la cámara) — el `int()` que arma las filas a insertar no estaba protegido contra eso. **Fix:** esas filas se descartan antes de insertar (`dropna(subset=["id_product", "id_cliente"])`) — no se pueden alertar de todos modos, no hay forma de decir de qué producto se trata. Impacto real medido: 510 filas de `BALANCES_TOTALES` con `id_product` nulo en los últimos 30 días.

Estadística de "urgentes" inflada en la respuesta del recálculo. El primer resultado real mostró `urgentes: 285` en la respuesta de `/recalcular`, pero el listado (`GET /alertas`) solo mostraba 79. Causa: el conteo se hacía sobre el universo completo de balances evaluados (incluyendo "Quiebre" urgentes, que a propósito nunca se persisten — sección 4.3), no sobre lo que de verdad quedó insertado. **Fix:** el conteo ahora se hace sobre las mismas filas que se insertan, para que el número coincida con lo que el listado devuelve.

Normalización defensiva del día de visita. `RUTA_PROGRAMACION.dia` tiene variantes de codificación reales en producción: "Miércoles" (2.528 filas) convive con "Miercoles" sin tilde (433 filas), y "Sábado" (241) con "Sabado" (114). La función `_normalizar_dia()` compara solo los 2 primeros caracteres en mayúsculas ("MI", "SA"), así que ambas variantes matchean igual sin necesidad de limpiar el dato en la base — validado contra estos valores reales antes de dar el módulo por terminado.

8. Dependencias de datos e integridad a validar

Igual que en Plan de Acción, esta tabla resume qué información usa el módulo y qué hay que auditar antes de confiar en el resultado al 100%:

TABLA / CAMPO	PARA QUÉ SE USA	QUÉ HAY QUE VALIDAR
<code>FRECUENCIAS_PDVS_CLIENTE.frecuencia_semanal</code>	Define <code>bucket_frecuencia</code> — agrupa la línea base (Capa 1) por cadencia de visita real	CRÍTICO Cobertura real medida: 388 de 8.046 combinaciones (PDV, cliente) activas — 4,8%. El 95,2% restante cae en el default (<code>fillna(1.0)</code> , bucket "1-2/semana") sin importar su cadencia real. Es el mismo hueco de datos que ya afecta a Plan de Acción (ver su ficha, sección 7).
<code>RUTA_PROGRAMACION.dia</code> (activa=1)	Calcula <code>días_hasta_próxima_visita</code> para el ajuste de urgencia (sección 5)	Prácticamente completo (14.599 filas activas). Variantes de codificación ya manejadas defensivamente (sección 7). 1 fila con <code>dia</code> vacío.
<code>PUNTOS_INTERES1.jerarquia_nivel_2</code>	Define el tipo de PDV para agrupar la línea base	OK 100% completo (5.200 de 5.200 puntos de venta).
<code>BALANCES_TOTALES.id_product</code>	Identifica de qué producto es cada balance	510 filas/mes sin identificar (~ver sección 7) — se descartan sin romper

el cálculo, pero esas visitas pierden cobertura de alerta.

RUTA_PROGRAMACION.prioridad	Reservado para ordenar alertas (sección 5) — todavía no implementado	Limpio hoy (solo "Alta"/"Media"/"Baja", sin valores sueltos) — a diferencia de lo que reportaba la ficha de Plan de Acción en su momento.
-----------------------------	--	---

9. Qué sí hace el módulo hoy / qué NO hace todavía

Sí hace

1. Calcula una línea base de inventario "normal" por percentiles, específica por categoría, tipo de PDV, frecuencia y cliente.
2. Detecta tendencia de caída comparando las 2 lecturas más recientes de cada producto por PDV.
3. Clasifica riesgo (Normal / Medio / Alto) sin que la prioridad del PDV distorsione la detección.
4. Cruza contra la próxima visita programada para decidir si el riesgo es urgente de verdad.
5. Corre solo (línea base 1x/día, alertas cada 4h) y permite forzar el recálculo manualmente.
6. Tiene pantalla propia en el portal con KPIs, filtros (riesgo/urgencia/cliente) y tabla de alertas.

NO hace todavía

1. **NO** ordena las alertas por prioridad del PDV — solo por urgencia/riesgo/días (sección 5).
2. **NO** envía notificación (push/email/chat) al analista — la alerta solo se ve si entra al portal.
3. **NO** tiene forma de marcar una alerta como "atendida" o descartarla manualmente — cada recálculo reemplaza la tabla entera.
4. **NO** distingue un producto temporalmente sin stock de uno discontinuado — ambos generan la misma alerta.
5. **NO** está expuesto todavía al cliente ni al mercaderista (roles admin/analyst por ahora) — el diseño original contempla eso como evolución futura.

10. Resultados observados en producción

MÉTRICA	VALOR OBSERVADO (15 AGO 2026)
Balances evaluados en el último recálculo de Capa 2	29.654
Alertas activas (Riesgo ALTO + MEDIO)	773
De esas, urgentes (quiebra antes de la próxima visita)	78
Grupos de línea base calculados	146

Estos números cambian en cada recálculo automático — son datos reales, no de prueba, capturados el mismo día del despliegue inicial.

11. Referencia técnica

11.1 Endpoints

ENDPOINT	QUÉ HACE
POST /api/quiebre/linea-base/recalcular	Capa 1. Background por default, <code>sincrono=true</code> devuelve el resultado real
POST /api/quiebre/alertas/recalcular	Capa 2. Necesita línea base previa. Mismo patrón background/sincrono
GET /api/quiebre/alertas	Listado con filtros (riesgo, urgente, id_cliente, identificador_pdv)
GET /api/quiebre/linea-base/info	Fecha del último cálculo y cantidad de grupos

11.2 Dónde corre

Mismo patrón que Plan de Acción: jobs en `app/services/scheduler_service.py` sobre APScheduler, sin servicio nuevo que mantener. Línea base cada 24h (`QUIEBRE_LINEA_BASE_INTERVAL_HOURS`), alertas cada 4h (`QUIEBRE_ALERTAS_INTERVAL_HOURS`).

11.3 Glosario

TÉRMINO	SIGNIFICADO
P10 / P25	Percentil 10 / 25 de "caras" (facing) dentro de un grupo — no promedio
bucket_frecuencia	Agrupación de la frecuencia semanal en 4 rangos, no el valor exacto
Urgente	Riesgo ALTO/MEDIO que va a llegar a 0 antes de la próxima visita programada
Ventana expansiva	(N1, no acá) — mencionado por contraste; este módulo usa una ventana fija de 90 días

12. Qué hace falta antes de confiar al 100%

1. **Cargar FRECUENCIAS_PDVS_CLIENTE real** — hoy 4,8% de cobertura sobre combos activos (sección 8). Es la acción de mayor impacto: sin esto, la agrupación de la línea base usa una frecuencia asumida, no la contratada.
2. **Definir el proceso operativo:** quién revisa las alertas urgentes y con qué frecuencia — hoy no hay notificación activa, depende de que alguien entre al portal.
3. **(Recomendado)** Implementar el ordenamiento por prioridad de PDV en el listado (sección 5, diseñado pero no construido).
4. **(Recomendado)** Dejar correr el sistema unas semanas y comparar las alertas "Riesgo ALTO" contra los quiebres reales que efectivamente ocurrieron después, para calibrar si P10/P25 y el piso de 5 muestras son los umbrales correctos o necesitan ajuste.
5. **Capacitar al equipo** (analistas de cuenta) en la nueva pantalla antes de reemplazar el proceso en Excel.

Ficha técnica del módulo "Quiebre Dinámico" — Nexus2DI. Documento elaborado a partir del desarrollo real del módulo durante agosto de 2026, incluyendo pruebas e incidentes reales contra producción. Ante cualquier duda técnica adicional, contactar al equipo de desarrollo de Nexus2DI.